人工智能与深度学习

神经网络与深度学习（一）

华中科技大学 机械科学与工程学院
陈伟

2026

背景：手写数字识别问题

手写数字识别



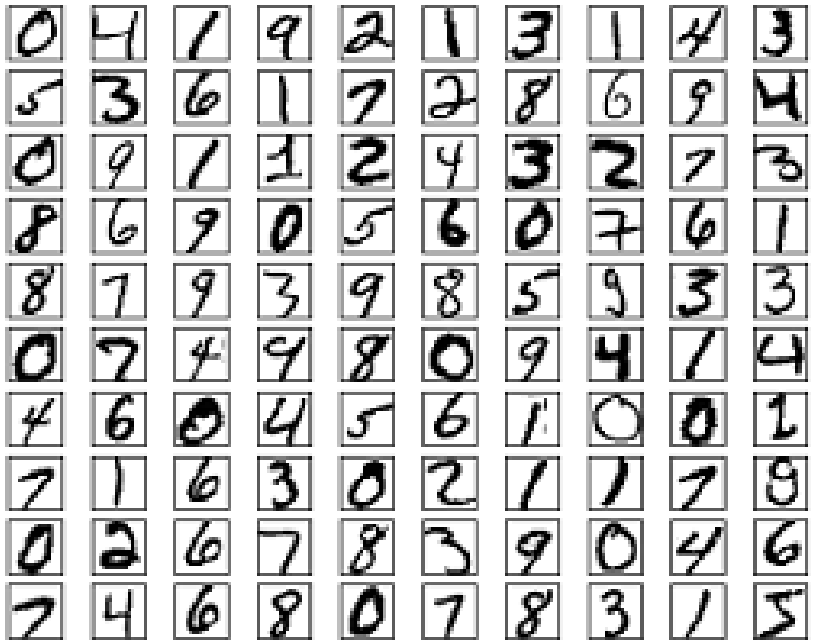
504192

- 这行数字：504192
- 每个数字是 28x28 个像素
- 如何通过机器学习识别这些手写数字?

手写数字识别

- 使用最普通的神经网络，我们只需要几十行代码，即可达到超过96%的准确率

- 经简单调整后，可到99%准确率

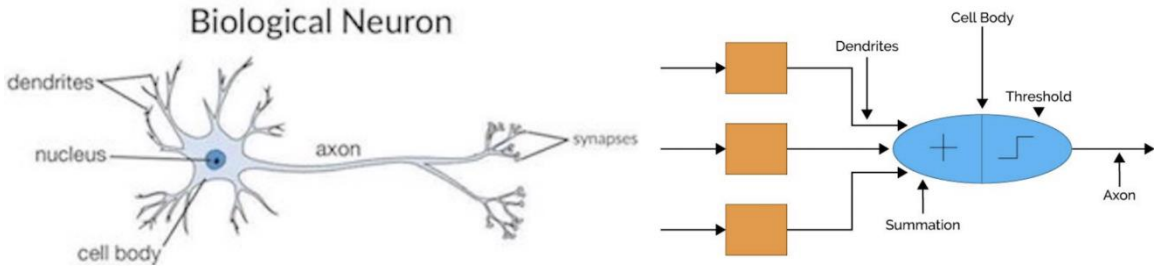


感知机（perceptron）

神经元（neuron）

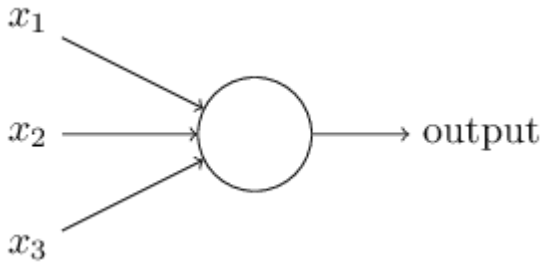
- **人工神经网络（artificial neural network）最初是受到生物神经网络的启发而设计的**

- **生物神经网络中最基本的单元是神经元**
  - 每个神经元与其它神经元相连
  - 当一个神经元被“激活”时，它会向相连的神经元发送化学物质，从而改变其它神经元内的电位
  - 如果神经元的电位超过了一个阈值，它就会被激活



感知机（perceptron）

- 感知机是一种神经元。它有多个二元（binary）输入 $x_1, x_2, \ldots$ 和一个二元输出 output



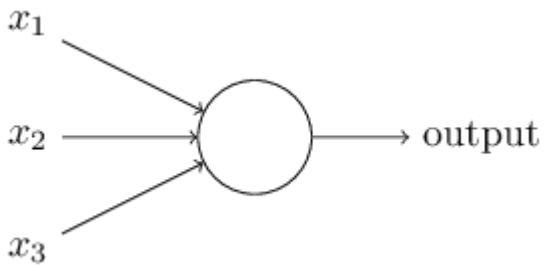
- 感知机如何根据输入决定输出呢？对于每个输入，有一个相应的权重，$w_1, w_2, \ldots$。假如加权和 $\sum_j w_j x_j$ 小于某个阈值，就输出 0；反之，则输出1.

$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

感知机：示例

- **假如你想决定今天是否去某个公园玩，你考虑以下三个因素**
  - 今天天气好不好
  - 你的男朋友/女朋友今天是否想和你一起去
  - 到那个公园交通方不方便

- 我们用$x_1 = 1$表示天气好，$x_1 = 0$表示天气不好。以此类推。

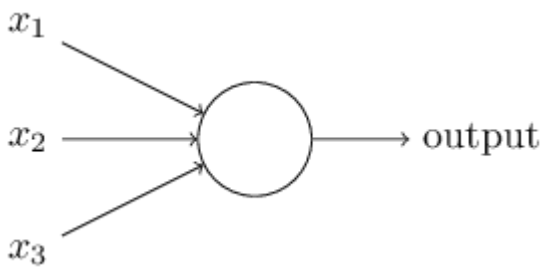


感知机：示例

- 我们来用感知机模型做决定。

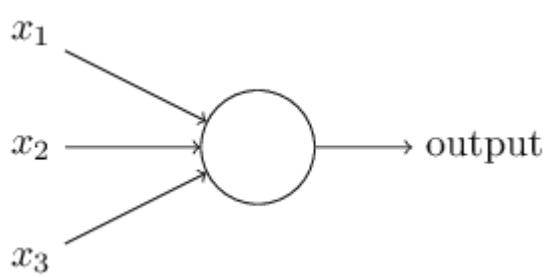
  可能你在出去玩的时候对天气特别看重，于是可以令 $w_1 = 6$，$w_2 = 3$, $w_3 = 2$。然后确定阈值为 4。

  在确定了权重和阈值后，感知机就可以根据输入来计算输出了。



感知机：示例

- $w_1 = 6$，$w_2 = 3$, $w_3 = 2$；阈值为 4。

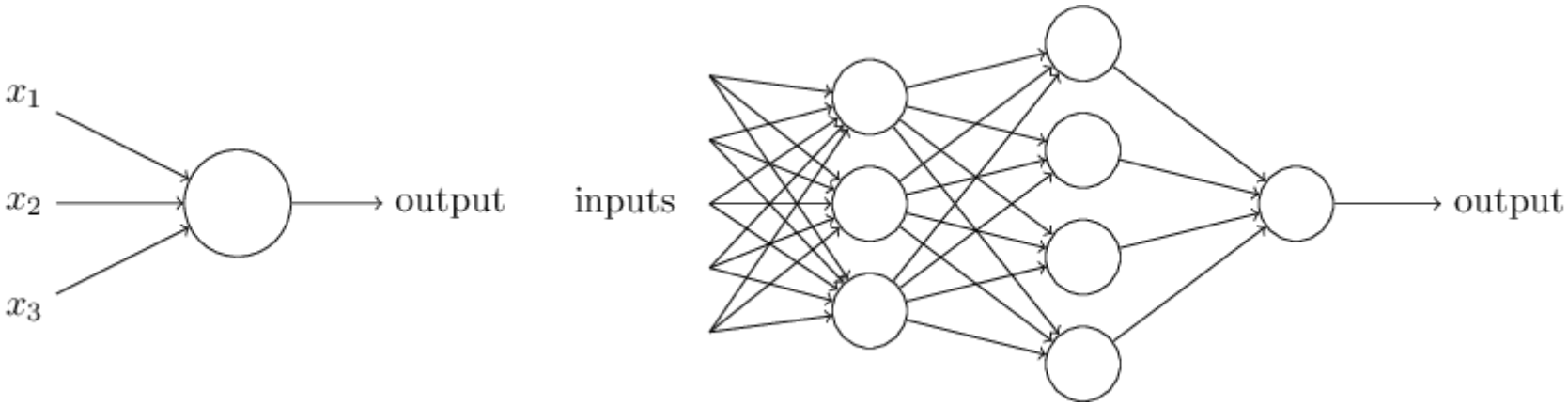


- 假如天气好，$\sum_j w_j x_j = 6 \times 1 + \ldots > 4$，输出为1：出去玩

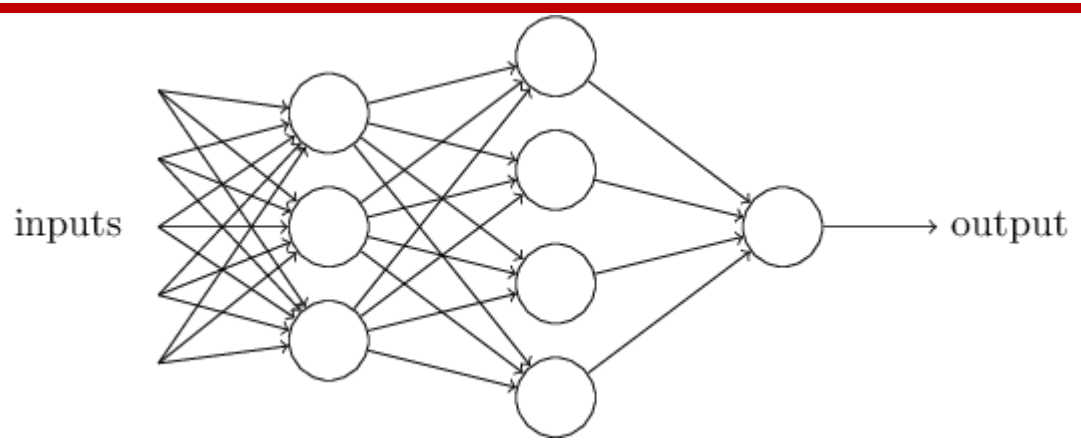
  假如天气不好，但朋友愿意陪你，且交通方便 $\sum_j w_j x_j = 6 \times 0 + 3 \times 1 + 2 \times 1 = 5 > 4$，输出为1：出去玩

感知机与神经网络

- 示例说明了在感知机中不同的权重如何影响输出结果
- 显然，用一个感知机来模拟人类决策过程，有点过于简单了
- 我们可以将多个感知机连接成复杂的神经网络



神经网络



- **上图网络的第一层（3个感知机）依据5个输入做三个决策（3个输出）**
- **第二层（4个感知机）依据第一层的决策输出，再进行决策**
- **第三层（1个感知机）依据第二层的结果，确定最终的输出**
  - 注：感知机之间传递的信号都是 0 或 1

感知机

- 我们之前把感知机模型写作

$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

现在，我们使用新的符号简化一下 $\sum_j w_j x_j \leq \text{threshold}$

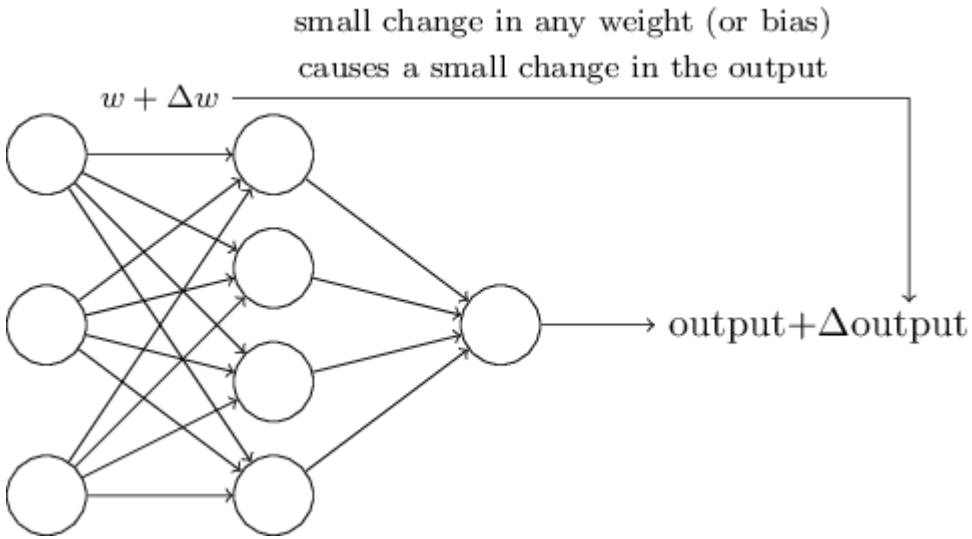
首先，使用 m 维向量 $\boldsymbol{x}$ 和 $\boldsymbol{w}$；另外，引入 $b = -\text{threshold}$,

b 被称为偏置（bias）. 简化后

$$\text{output} = \begin{cases} 0 & \text{if } \boldsymbol{w}^T \boldsymbol{x} + b \leq 0 \\ 1 & \text{if } \boldsymbol{w}^T \boldsymbol{x} + b > 0 \end{cases}$$

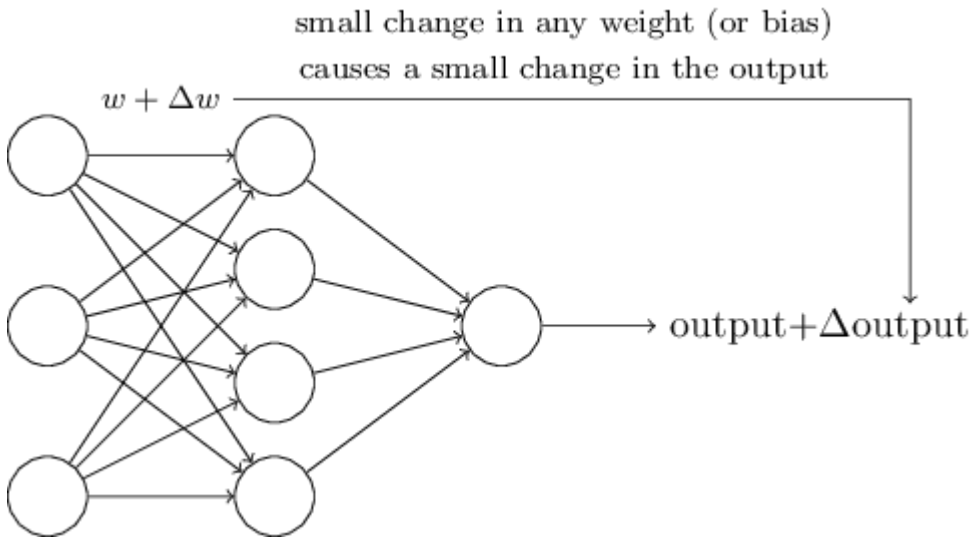
感知机

- 神经网络训练，就是让神经网络每个神经元学习得到最优的 $\boldsymbol{w}$ 和 b
- 若要学习 $\boldsymbol{w}$ 和 b，就需要知道 $\boldsymbol{w}$ 和 b 的变化和输出的变化之间的关系



感知机

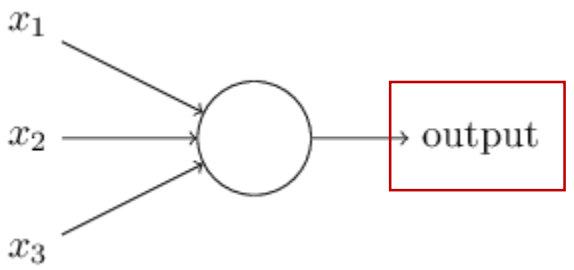
- 我们希望 w 和 b 的微小变化，也引起输出的微小变化（而不是跳变）
- 从数学上，我们也希望输出对于 w 和 b 可导。而目前这种基于阈值的感知机无法做到这一点



Sigmoid **神经元**

Sigmoid 神经元

- 我们基于只考虑（多个）二元输入与二元输出的感知机，来引入Sigmoid神经元
- 和感知机一样，Sigmoid神经元仍然有 $\boldsymbol{x} = [x_1, x_2, ...]^T$ 等多个输入，但其输入不再只是0或1；Sigmoid神经元的输入可以是0到1之间的任意值，比如0.638

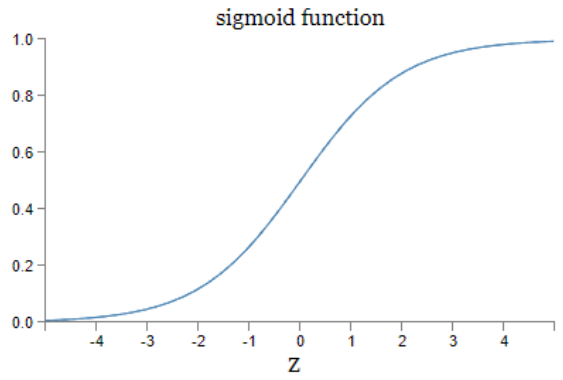


Sigmoid 神经元

- Sigmoid神经元仍然有权重参数 $\boldsymbol{w}$ 和偏置参数 b

- Sigmoid神经元的输出也不再只是0或1，而是 $\sigma(\boldsymbol{w}^T\boldsymbol{x}+b)$，其中 σ 就是 Sigmoid函数，其形式为

$$\sigma(z)=\frac{1}{1+e^{-z}}.$$



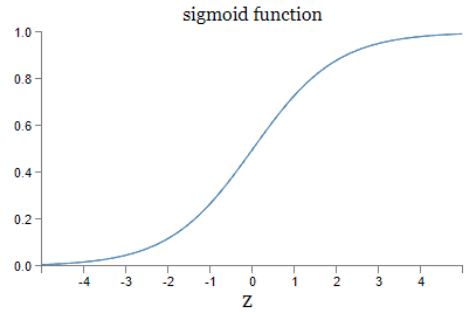
因此，Sigmoid神经元的输出就是

$$\frac{1}{1+e^{-\boldsymbol{w}^T\boldsymbol{x}-b}}$$

Sigmoid 神经元

- 表面上看，Sigmoid神经元似乎和感知机差别很大。其实， Sigmoid 神经元和感知机很接近
- 若 $z = \boldsymbol{w}^T\boldsymbol{x} + b \gg 0$，那么 $e^{-z} \approx 0$ 所以 $\sigma(z) \approx 1$. 此时 Sigmoid神经元的输出十分接近1，即十分接近感知机的输出
- 若 $z = \boldsymbol{w}^T\boldsymbol{x} + b \ll 0$，那么 $e^{-z} \to \infty$ 所以 $\sigma(z) \approx 0$. 此时 Sigmoid神经元的输出十分接近0，也十分接近感知机的输出

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

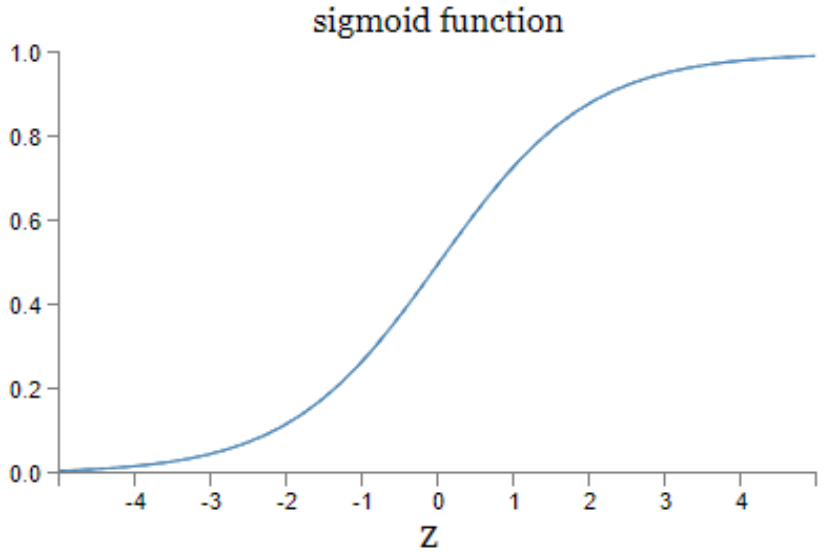


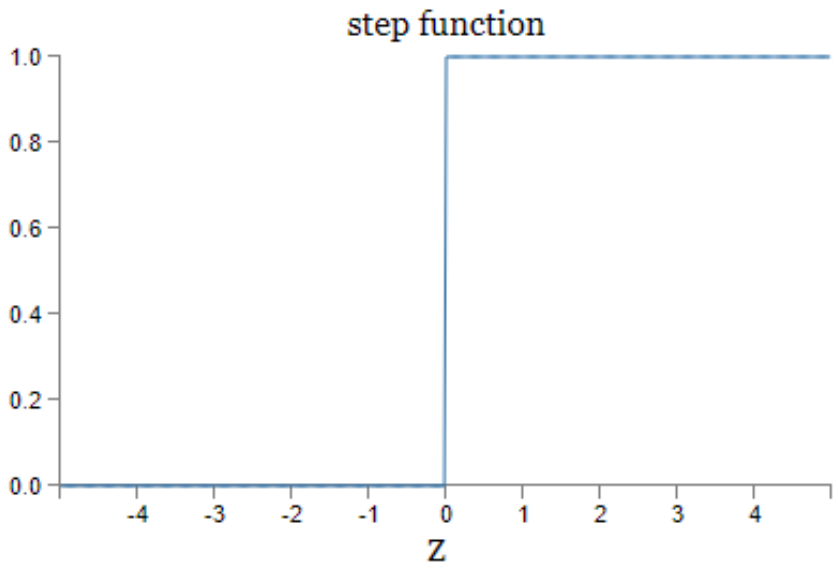
Sigmoid 神经元

- 只有当 $z = \boldsymbol{w}^T\boldsymbol{x} + b$不太大也不太小时，Sigmoid神经元和感知机的输出有明显差异

- **Sigmoid函数可以看作是对阶跃函数的“平滑版”**

  如果将Sigmoid神经元中的 σ 换成阶跃函数，其输出就和感知机完全一样了





Sigmoid 神经元

- 事实上，选择一个平滑的 σ 是我们当前的主要目的。至于 σ 具体是什么形式，对我们来说暂时不重要。

- 平滑代表了参数的微小变化 Δw_j 和 Δb 会引起神经元输出的微小变化

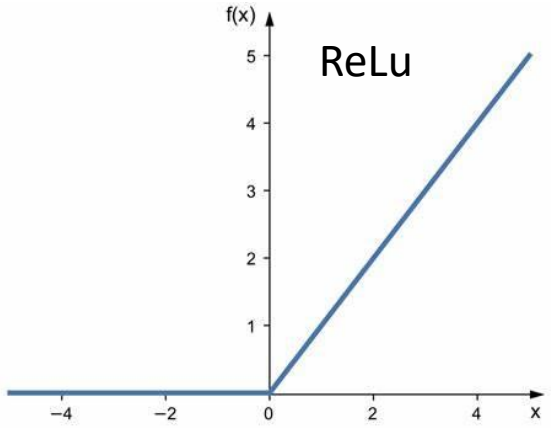
$$\Delta \text{output} \approx \sum_j \frac{\partial \text{ output}}{\partial w_j} \Delta w_j + \frac{\partial \text{ output}}{\partial b} \Delta b$$

上式也可以理解成，输出的微小变化 Δoutput 近似是 Δw_j 和 Δb 的线性组合；线性组合系数由这些偏导决定

Sigmoid 神经元

- Sigmoid 神经元用 $\sigma(\boldsymbol{w}^T\boldsymbol{x}+b)$来决定输出。我们将来还会考虑使用其它的函数 $f(\boldsymbol{w}^T\boldsymbol{x}+b)$ 来决定神经元的输出

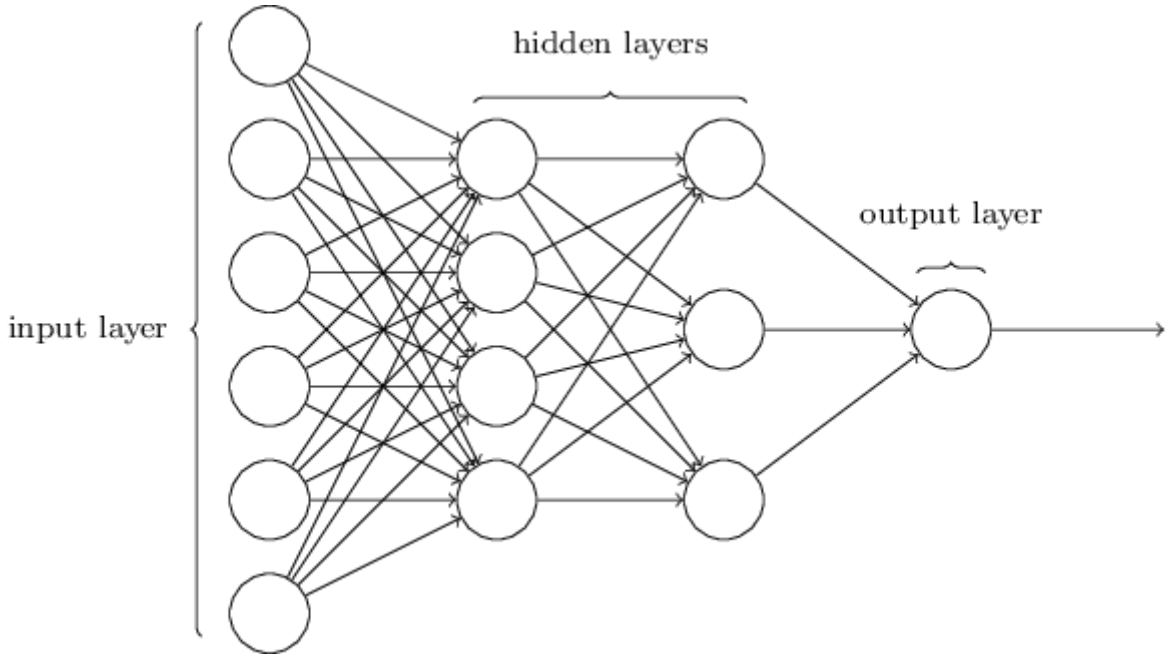
- 这时的 $f(\cdot)$ 就叫“激活函数”(activation function). Sigmoid 函数是常用的激活函数。其它常用激活函数还包括 ReLu（Rectified Linear Unit）



用神经网络识别手写数字（一）

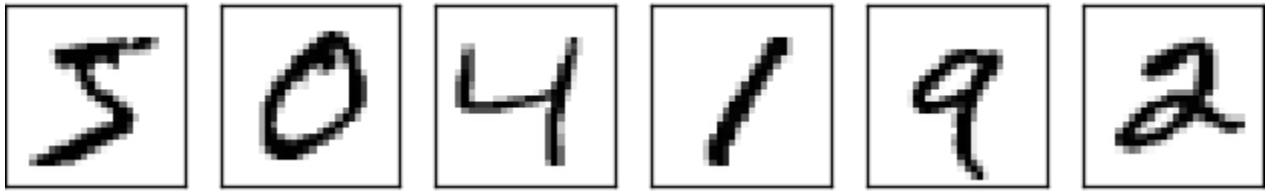
神经网络结构中的术语

- **输入层，输入神经元**
- **输出层，输出神经元**（图中仅1个输出神经元，也可有多个）
- **隐藏层（hidden layer)**（图中有2个隐藏层）



手写数字识别

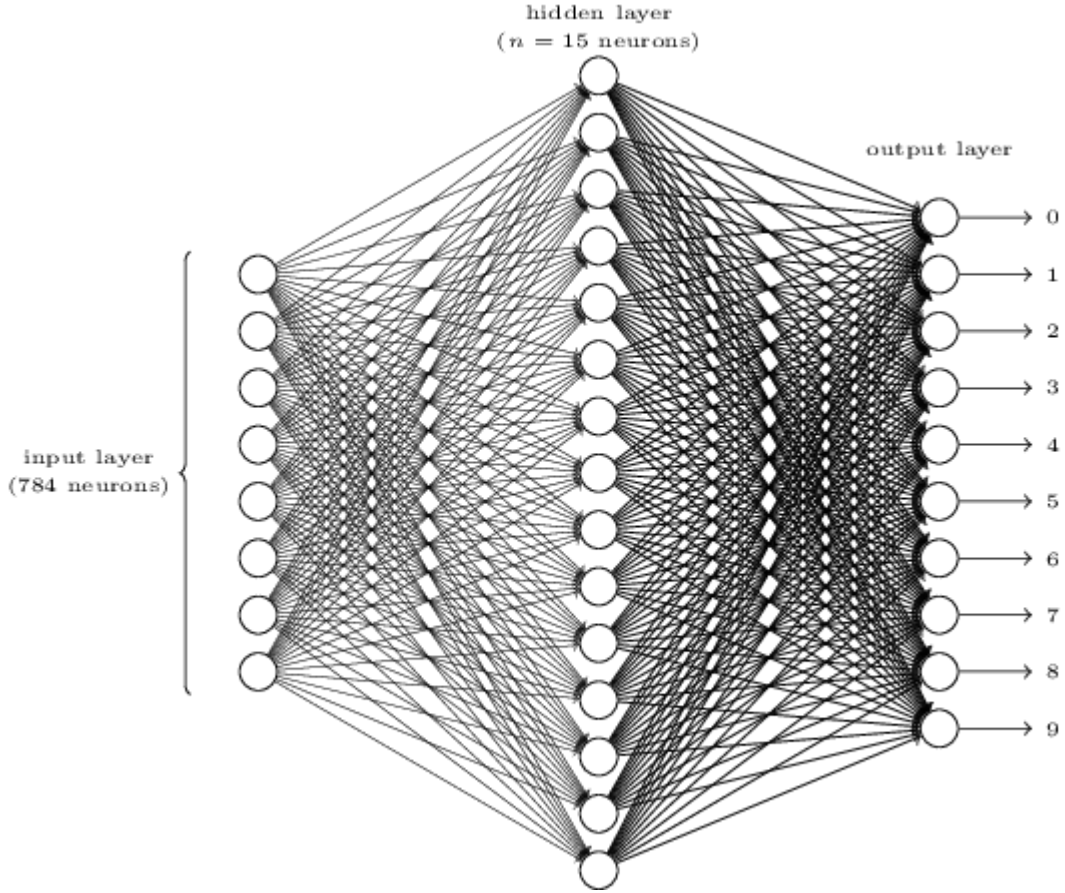
- **我们考虑逐一识别手写数字**



- **每个数字，是28像素x28像素的图像** 
  - 输入的图像为灰度图像，每个像素0.0代表白，1.0代表黑，0.0和1.0之间的数字代表相应的灰色
  - 每幅图像对应一个28x28=784维的向量
- **我们设计神经网络模型来实现0~9手写数字的识别**

神经网络结构

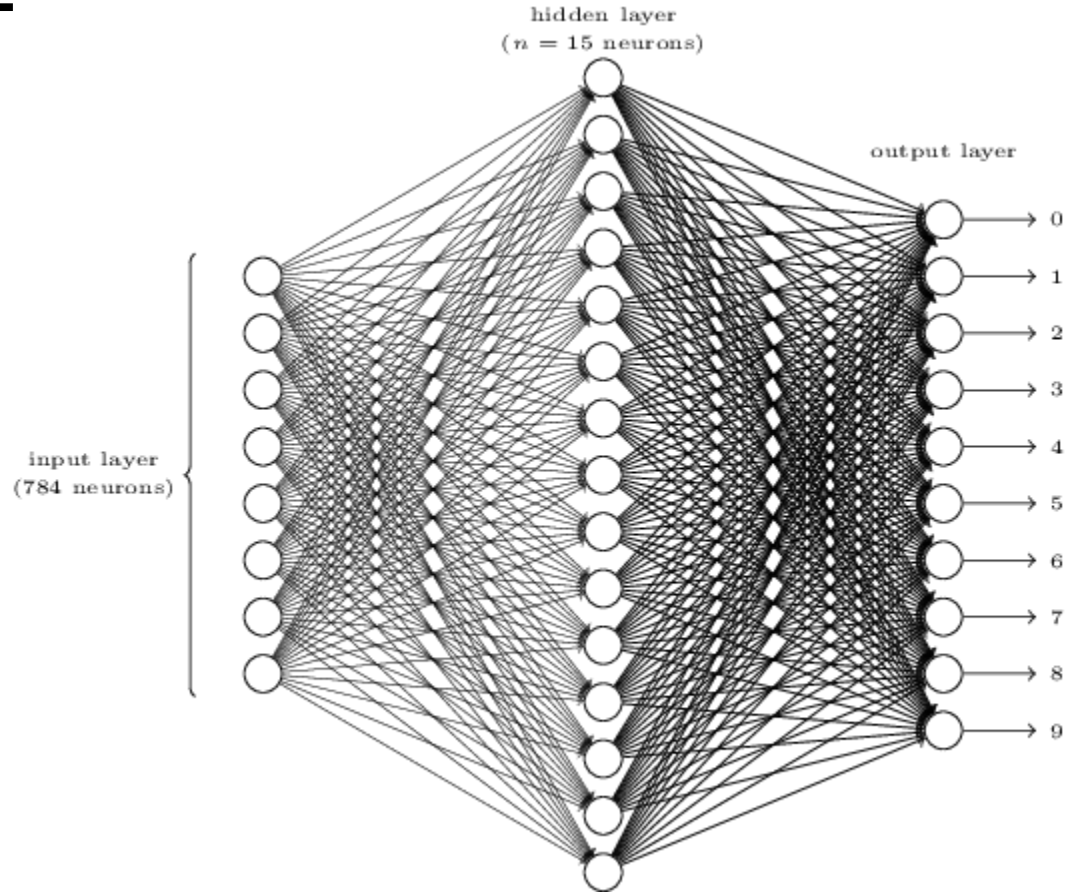
- 建立一个三层神经网络，其中包括一个隐藏层



注：输入层有28x28=784个神经元，图中没有全部画出来（只画了8个示意）
隐藏层有15个神经元，输出层有10个神经元

神经网络结构

- 输出层有10个神经元，其中若第一个神经元被激活了，其输出接近1，则我们认为这个数字是0，以此类推



神经网络结构

- 神经网络的输入 $x \in \mathbb{R}^{784}$，我们把对应 x 的真实的标签写作 $y(x),\ y(x) \in \mathbb{R}^{10}$.
- 若 x 代表的图片是数字 6，那么我们期望的输出是 $y(x) = [0,0,0,0,0,0,1,0,0,0]^T$.
- 我们把 x 经过神经网络后的实际输出记为 a，$a \in \mathbb{R}^{10}$. a 可以会出现不完全等于0或1的元素，如 $a = [0, 0.1, 0, 0, 0, 0.01, 0, 0.85, 0, 0]$.

建立优化问题，计算神经网络参数

- 我们需要通过找到合适的神经网络中每个神经元的参数 $\boldsymbol{w}$ 和 b，来使神经网络的输出 a 和真实的标签 $y(x)$ 尽量接近。

  和其它机器学习训练算法一样，我们要设计损失函数，建立一个优化问题，来求解最优的 $\boldsymbol{w}$ 和 b

损失函数

- 考虑如下损失函数

$$C(\boldsymbol{w}, b) = \frac{1}{2n} \sum_x ||y(x) - a||^2$$

其中 n 是训练中样本的数量，a 的值取决于 x, $\boldsymbol{w}$ 和 b（所以也可以写成 $a(x, \boldsymbol{w}, b)$）

C 是二次型，因此它是非负的

$C(\boldsymbol{w}, b) \approx 0$ 说明 a 很接近 $y(x)$，即神经网络的识别效果好

神经网络的训练

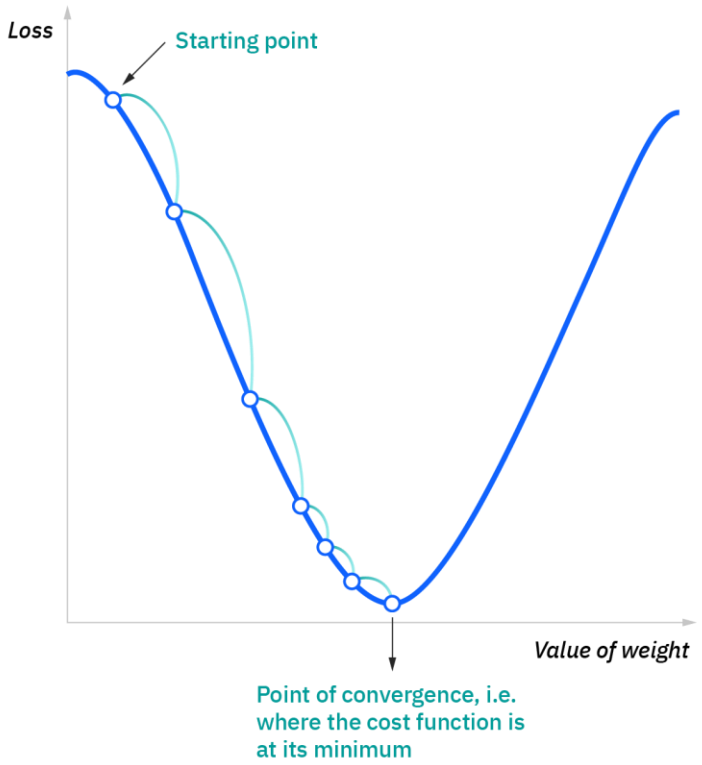
- 有了大量样本，和损失函数，如何训练神经网络的参数

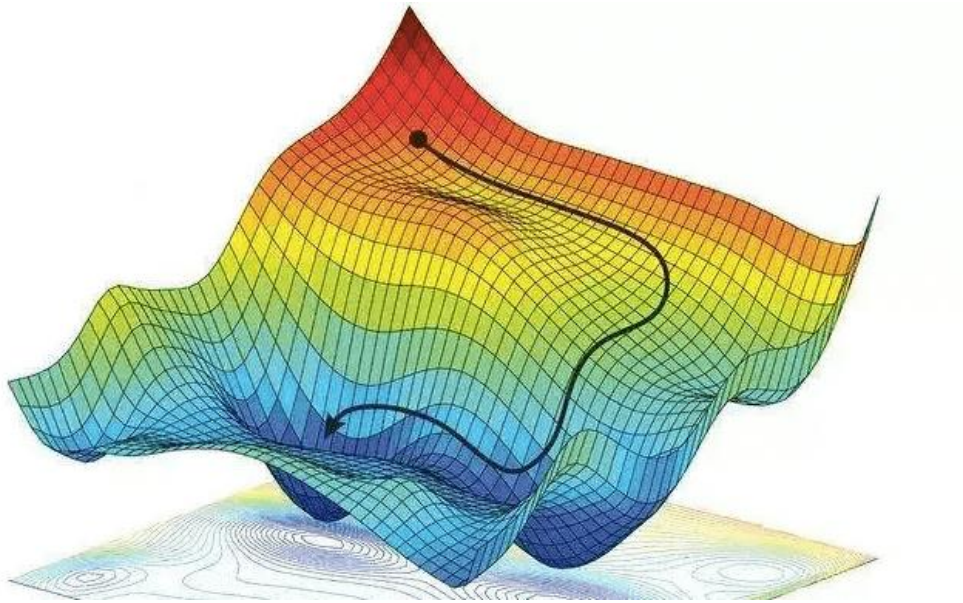
- 神经网络的参数训练都是基于梯度下降方法

梯度下降
(Gradient Descent)

优化算法：梯度下降

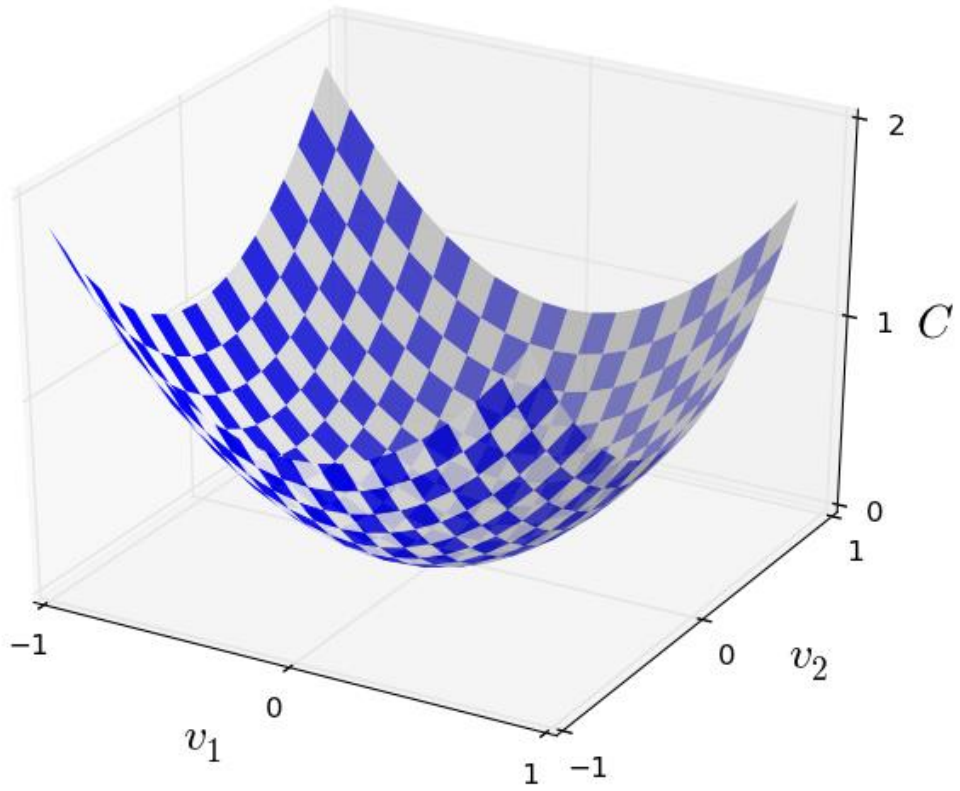
- 我们使用“梯度下降”方法来找 C 的最小值，和使其最小的神经网络参数 $\boldsymbol{w}$ 和 b





优化算法：梯度下降

- 假设我们要最小化某个函数 $C(\boldsymbol{v})$，$\boldsymbol{v} = [v_1, v_2, ...]$, $C \in \mathbb{R}$.



我们想在图中的曲面上找一系列的点，它们的 C 值不断下降

图示例子中，$\boldsymbol{v} \in \mathbb{R}^2$

优化算法：梯度下降

- 假设我们已知了某个点 (v_1, v_2, C)，首先，我们想知道如果对 $\boldsymbol{v}$ 作微小的改变，C 会如何变化

  考虑关于 $\boldsymbol{v}$ 的如下改变：$v_1 \to v_1 + \Delta v_1$，$v_2 \to v_2 + \Delta v_2$，那么

$$\Delta C \approx \frac{\partial C}{\partial v_1}\Delta v_1 + \frac{\partial C}{\partial v_2}\Delta v_2$$

  我们想找到一组更好的参数 $\boldsymbol{v}'$，使得新参数下的 C 更小（即让 ΔC 为负值）

优化算法：梯度下降

- 定义 $\Delta \boldsymbol{v} = [\Delta v_1, \Delta v_2]^T$.

  定义 C 的梯度

$$\nabla C = \left[\frac{\partial C}{\partial v_1}, \frac{\partial C}{\partial v_2}\right]^T.$$

  那么

$$\Delta C \approx \frac{\partial C}{\partial v_1}\Delta v_1 + \frac{\partial C}{\partial v_2}\Delta v_2$$

  可以写成

$$\Delta C \approx \nabla C \cdot \Delta \boldsymbol{v}$$

优化算法：梯度下降

$$\Delta C \approx \nabla C \cdot \Delta \boldsymbol{v}.$$

如何选取 $\Delta \boldsymbol{v}$ 才能让 $\Delta C < 0$ 呢？

我们选取

$$\Delta \boldsymbol{v} = -\eta \nabla C,$$

其中 $\eta > 0$. 这样的话

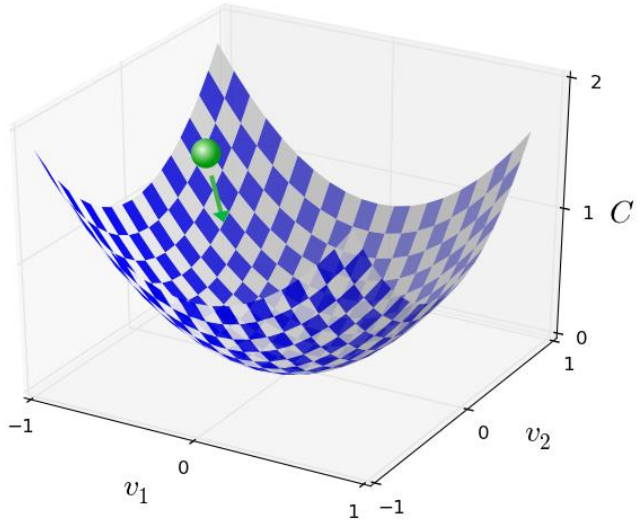
$$\Delta C \approx -\eta \nabla C \cdot \nabla C = -\eta ||\nabla C||^2 \leq 0$$

优化算法：梯度下降

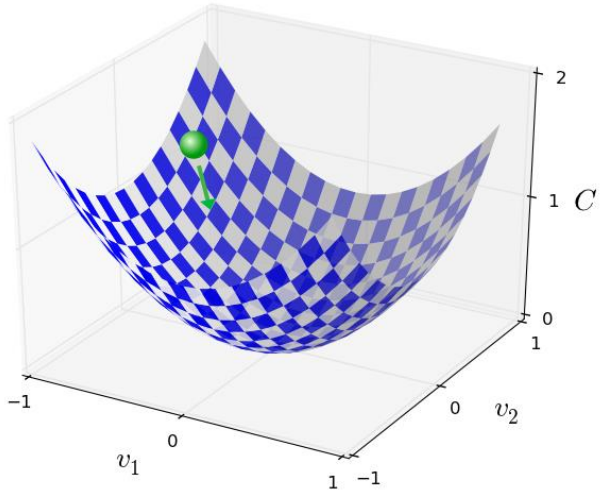
- 如果我们选取 $\Delta \boldsymbol{v} = -\eta \nabla C$, 那么 $\Delta C \approx -\eta ||\nabla C||^2 \leq 0$.

我们的参数需要按照以下方式改变

$$\boldsymbol{v} \to \boldsymbol{v}' = \boldsymbol{v} - \eta \nabla C.$$



优化算法：梯度下降



- $$\boldsymbol{v} \to \boldsymbol{v}' = \boldsymbol{v} - \eta \nabla C.$$

  η 是“学习率”（learning rate）

  若 η 太大，公式中的 $\approx$ 可能不再成立，会导致 $\Delta C > 0$；若 η 太小，ΔC 绝对值也会很小，导致“下降”很慢，需要很长时间才能收敛到最优值

  我们一步一步地更新参数，损失函数就会一步步地接近“谷底”

在神经网络中使用梯度下降

- 在神经网络中，我们希望最小化的损失函数如下

$$C(\boldsymbol{w}, b) = \frac{1}{2n} \sum_x ||y(x) - a||^2$$

我们的参数 $\boldsymbol{v}$ 其实就是 $\boldsymbol{w}$ 和 b.

因此，基于梯度下降的参数更新公式为

$$\boldsymbol{w} \to \quad \boldsymbol{w}' = \boldsymbol{w} - \eta \frac{\partial C}{\partial \boldsymbol{w}}$$

$$b \to \quad b' = b - \eta \frac{\partial C}{\partial b}$$

在神经网络中使用梯度下降

- 在具体执行时，梯度下降方法有很多变体

- 在这里我们考虑其中一种最重要的：**随机梯度下降**（stochastic gradient descent，SGD）

随机梯度下降

- 神经网络训练中的损失函数，是关于所有训练样本的误差的均值

$$C(\boldsymbol{w}, b) = \frac{1}{2n} \sum_x ||y(x) - a||^2$$

我们令

$$C_x = \frac{||y(x) - a||^2}{2},$$

那么

$$C = \frac{1}{n} \sum_x C_x.$$

随机梯度下降

- 若要计算 ∇C，我们需要对于 n 个训练样本单独计算 ∇C_x，再求其均值。通常训练样本数量都是很大的，这样做效率会很低。

- 随机梯度下降方法就是针对这个问题而设计的。在随机梯度下降的每一步中，我们不使用所有训练样本去计算 ∇C，而是从训练集中随机选取一小部分样本，计算这一小部分样本的 ∇C_x 平均值，把这个值当作对真实 ∇C 的估计

- 随机梯度下降可以加速对梯度下降的计算，从而加速神经网络的训练

随机梯度下降

- 具体而言，在随机梯度下降方法中，每次我们从所有训练样本中随机抽取 m 个。我们把这些样本记为 $\boldsymbol{X}_1, \boldsymbol{X}_2, ..., \boldsymbol{X}_m$，这样的一组 m 个样本被称为 mini-batch

  我们用一个 mini-batch 中的 m 个样本来近似估计 ∇C,

$$\frac{\sum_{j=1}^{m} \nabla C_{\boldsymbol{X}_j}}{m} \approx \frac{\sum_x \nabla C_x}{n} = \nabla C$$

用mini-batch中的样本计算

用所有样本计算

随机梯度下降

- 我们用 mini-batch 来估计梯度

$$\nabla C \approx \frac{\sum_{j=1}^{m} \nabla C_{\boldsymbol{X}_j}}{m}$$

在更新神经网络参数时，使用基于 mini-batch的梯度估计

$$\boldsymbol{w} \rightarrow \quad \boldsymbol{w}' = \boldsymbol{w} - \frac{\eta}{m} \sum_j \frac{\partial C_{\boldsymbol{X}_j}}{\partial \boldsymbol{w}}$$

$$b \rightarrow \quad b' = b - \frac{\eta}{m} \sum_j \frac{\partial C_{\boldsymbol{X}_j}}{\partial b}$$

随机梯度下降

- 在参数更新之后，我们进入下一步：再选取一个新的 mini-batch，并用其更新参数

  若每次 mini-batch 选取的样本不重复（即无放回抽样），所有样本都被选过一次后，我们说训练完成了一个 epoch

用神经网络识别手写数字（二）

MNIST数据集

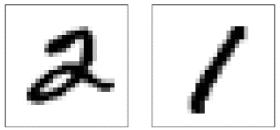
- 手写数字数据来自于 MNIST 数据集，其中60000张图像数据用于训练，10000张图像数据用于测试

- 60000张用于训练的图像被分为两部分，50000张为训练集，10000张为验证集

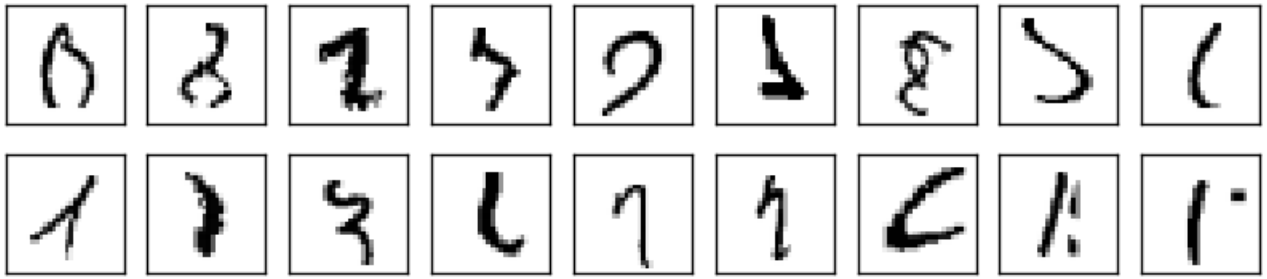
基准模型

- **神经网络模型的表现有多好?**

- **MNIST 测试集上的准确率**



  - 22.25%：基于灰度的方法（仅看图像有多少部分是黑的）
  - 94.35%：直接使用 scikit-learn 默认的 SVM
  - 98.5%：调整过的 SVM
  - **99.79%：神经网络**，10000张图识别错误21张，超过了人类的表现（MNIST 中有一些难以辨认的图像）



神经网络反向传播算法

反向传播（backpropagation）

- 在神经网络中，基于随机梯度下降的参数更新公式为

$$\boldsymbol{w} \to \quad \boldsymbol{w}' = \boldsymbol{w} - \frac{\eta}{m} \sum_j \frac{\partial C_{\boldsymbol{X}_j}}{\partial \boldsymbol{w}}$$

$$b \to \quad b' = b - \frac{\eta}{m} \sum_j \frac{\partial C_{\boldsymbol{X}_j}}{\partial b}$$

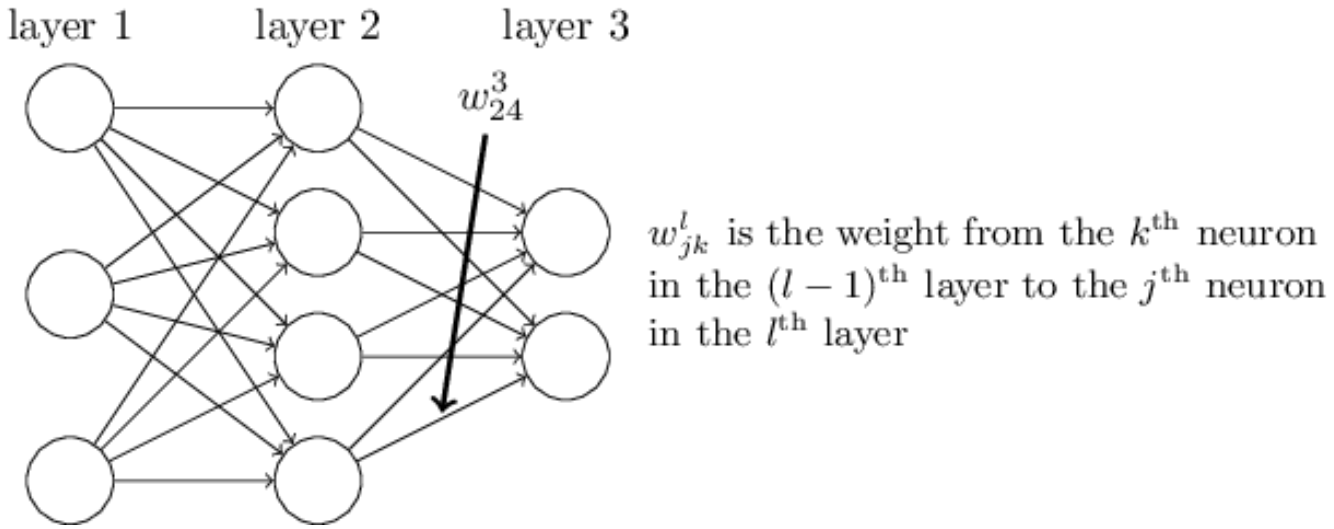
- 给定神经网络及 $C(\boldsymbol{w}, b)$, 如何计算 $\frac{\partial C}{\partial \boldsymbol{w}}$ 和 $\frac{\partial C}{\partial b}$?



反向传播算法

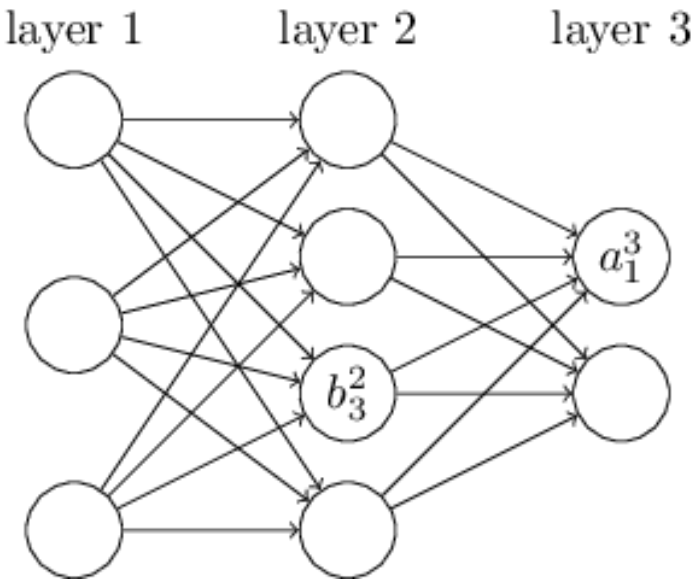
用向量和矩阵形式表示神经网络

- 我们用 w_{jk}^{l} 表示连接第 $(l-1)$ 层的第 k 个神经元，与第 l 层的第 j 个神经元之间的权重



用向量和矩阵形式表示神经网络

- 我们用 b_j^l 表示第 l 层第 j 个神经元的偏置；用 a_j^l 表示第 l 层第 j 个神经元的输出信号



$$a_j^l = \sigma\left(\sum_k w_{jk}^l a_k^{l-1} + b_j^l\right)$$

用向量和矩阵形式表示神经网络

- $$a_j^l = \sigma\Big(\sum_k w_{jk}^l a_k^{l-1} + b_j^l\Big)$$

我们把上式写成向量形式：

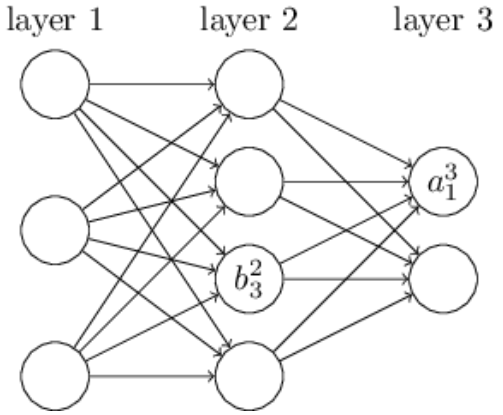
$$\boldsymbol{a}^l = \sigma(\boldsymbol{w}^l \boldsymbol{a}^{l-1} + \boldsymbol{b}^l),$$

其中，$\boldsymbol{a}^l$ 是第 l 层所有神经元输出组成的向量，$\boldsymbol{w}^l$ 是第 l 层所有神经元权重参数组成的矩阵，$\boldsymbol{b}^l$ 是第 l 层所有神经元偏置参数组成的向量；激活函数 $\sigma(\cdot)$ 为对每个元素的操作，比如

$$\sigma\left(\begin{bmatrix}2\\3\end{bmatrix}\right) = \begin{bmatrix}\sigma(2)\\\sigma(3)\end{bmatrix}$$

用向量和矩阵形式表示神经网络

$$\boldsymbol{a}^l = \sigma(\boldsymbol{w}^l \boldsymbol{a}^{l-1} + \boldsymbol{b}^l)$$



- 向量和矩阵表示的神经网络更直观地告诉我们，激活函数是如何作用在层与层之间的

用向量和矩阵形式表示神经网络

- 我们定义

$$\boldsymbol{z}^l = \boldsymbol{w}^l \boldsymbol{a}^{l-1} + \boldsymbol{b}^l$$

$\boldsymbol{z}^l$ 称为第 l 层的加权输入（weighted input）。那么

$$\boldsymbol{a}^l = \sigma(\boldsymbol{z}^l)$$

通过观察可知，$\boldsymbol{z}^l$ 中的每个元素满足

$$z_j^l = \sum_k w_{jk}^l a_k^{l-1} + b_j^l$$

对代价函数的假设

- 反向传播的目标，是对神经网络中的任意一个神经元的参数 $\boldsymbol{w}$ 和 b，计算 $\frac{\partial C}{\partial \boldsymbol{w}}$ 和 $\frac{\partial C}{\partial b}$

  为此，我们需要对损失函数 C 做两个假设

对代价函数的假设

- 回顾：我们之前使用的损失函数如下

$$C = \frac{1}{2n} \sum_{x} ||\boldsymbol{y}(x) - \boldsymbol{a}^L(x)||^2$$

n 是样本总数，求和是针对所有的样本，$\boldsymbol{y}(x)$ 是对样本 x 的标签，$\boldsymbol{a}^L$ 是神经网络最后一层的输出，L 是神经网络的总层数。

$\boldsymbol{y}$ 和 $\boldsymbol{a}$ 可以是向量（参见之前的手写数字识别例子）

对代价函数的假设（一）

- 对代价函数的第一个假设是，它能被写成$C = \frac{1}{n}\sum_x C_x$这种形式，其中 C_x 是关于单个样本 x 的代价，C 是所有单个样本代价 C_x 的平均值

例如，之前使用的损失函数

$$C = \frac{1}{2n}\sum_x ||\boldsymbol{y}(x) - \boldsymbol{a}^L(x)||^2$$

之中，

$$C_x = \frac{1}{2}||\boldsymbol{y}(x) - \boldsymbol{a}^L(x)||^2$$

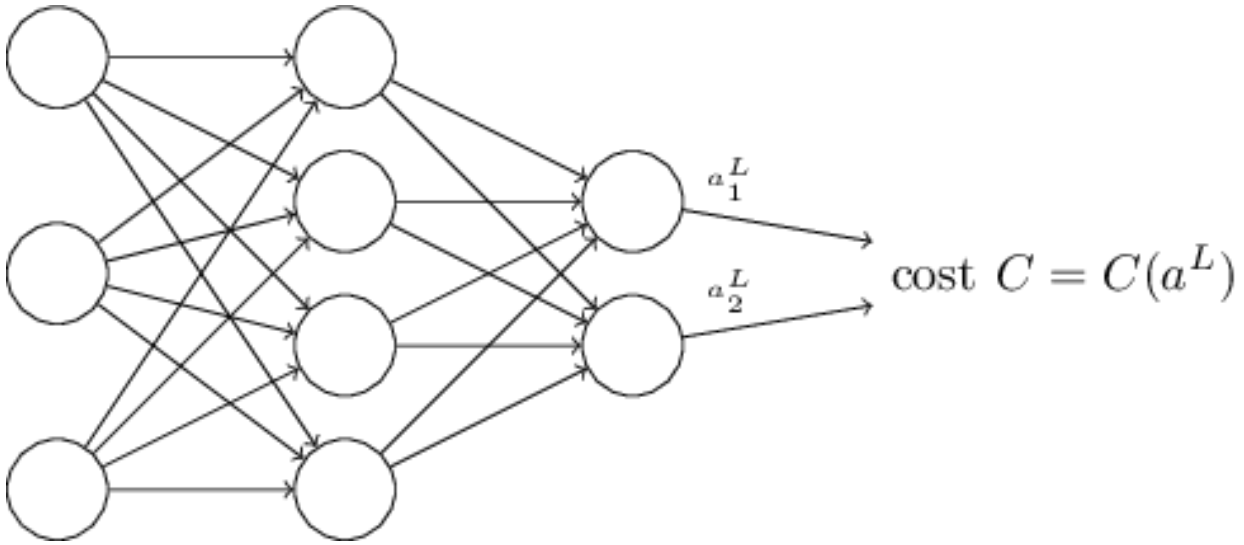
对代价函数的假设（一）

- 我们需要这个假设的原因是，在进行反向传播计算时，我们实际可以算的是针对每个样本 x 的偏导 $\frac{\partial C_x}{\partial \boldsymbol{w}}$ 和 $\frac{\partial C_x}{\partial b}$；因此需要总的损失 C 是关于各样本损失 C_x 的平均值

  基于这个原因，接下来我们假设样本 x 是固定不变的，这样可以暂时将 C_x 简写为 C。

对代价函数的假设（二）

- 对代价函数的第二个假设是，它是关于神经网络输出 a^L 的函数



对代价函数的假设（二）

- 例如，之前使用的损失函数中

$$C = \frac{1}{2}||\boldsymbol{y} - \boldsymbol{a}^L||^2$$

在 x 固定的情况下，$\boldsymbol{y}$ 也是确定的；C 是关于 $\boldsymbol{a}^L$ 的函数

引入新的符号：哈达玛积（Hadamard Product）

- 假设 s 和 t 是具有同样维度的向量，我们用 $s \odot t$ 来表示它们对应元素乘积组成的向量。所以 $(s \odot t)_j = s_j t_j$，比如

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} \odot \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 1 \times 3 \\ 2 \times 4 \end{bmatrix} = \begin{bmatrix} 3 \\ 8 \end{bmatrix}$$

神经元的“误差量”

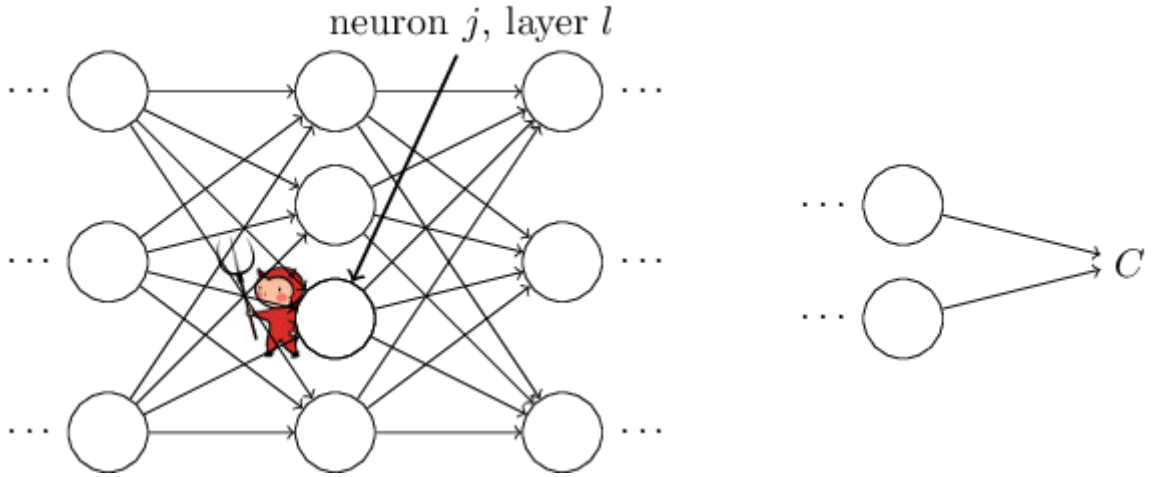
- 反向传播是为了理解每个神经元权重 $\boldsymbol{w}$ 与偏置 b 的变化是如何影响损失函数 C 的

  我们最终通过计算偏导 $\frac{\partial C}{\partial w^l_{jk}}$ 和 $\frac{\partial C}{\partial b^l_j}$ 来描述这种影响

  为此，我们引入一个中间量 δ^l_j（我们称其为第 l 层第 j 个神经元的“误差量”），来帮助计算

神经元的“误差量”

- 假设我们的神经网络中，有一个“小恶魔”，出现在了第 l 层第 j 个神经元处



这个小恶魔影响了神经元处理信息的过程：它在神经元加权输出 z_j^l 上增加了一个小变化，Δz_j^l，因此该神经元的输出变成了 $\sigma(z_j^l + \Delta z_j^l)$

神经元的“误差量”

- 收到小恶魔影响的这个神经元的输出，$\sigma(z_j^l + \Delta z_j^l)$，会随着神经网络继续向后传播，继而影响最终的损失函数 C。它引起的损失函数的变化是 $\frac{\partial C}{\partial z_j^l}\Delta z_j^l$

如果这个恶魔是个“好恶魔”，它会帮我们降低代价函数，即找到使 $\frac{\partial C}{\partial z_j^l}\Delta z_j^l$ 为负值的 Δz_j^l

神经元的“误差量”

- 若 $\frac{\partial C}{\partial z_j^l}$ 的绝对值比较大，小恶魔就可以选取一个相反符号的 Δz_j^l，使 $\frac{\partial C}{\partial z_j^l}\Delta z_j^l$ 为负值

  若 $\frac{\partial C}{\partial z_j^l}$ 的绝对值比较小，这时候小恶魔选取的 Δz_j^l（Δz_j^l 只是个微小变化）对 $\frac{\partial C}{\partial z_j^l}\Delta z_j^l$ 的影响不大。这时我们可以认为，这个神经元在当前网络中可能接近最优了

  我们可以粗略认为，$\frac{\partial C}{\partial z_j^l}$ 是对该神经元误差的度量，因此，定义

$$\delta_j^l = \frac{\partial C}{\partial z_j^l}$$

神经元的“误差量”

- $$\delta_j^l = \frac{\partial C}{\partial z_j^l}$$

我们用 δ^l 代表第 l 层的“误差量”组成的向量。

反向传播算法将会计算每一层的 δ^l，继而将它们与我们最终关心的 $\frac{\partial C}{\partial w_{jk}^l}$ 和 $\frac{\partial C}{\partial b_j^l}$ 联系起来

反向传播的四个重要公式（一）：最后一层→损失函数

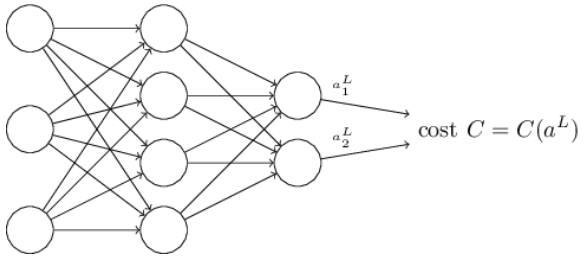
- 第一个公式：最终输出层的“误差量”

$$\delta_j^L = \frac{\partial C}{\partial z_j^L} = \frac{\partial C}{\partial a_j^L}\sigma'(z_j^L)$$

$\frac{\partial C}{\partial a_j^L}$ 表示了神经网络最后一层的输出的变化对代价函数的影响，$\sigma'(z_j^L)$ 表示了加权输出对经过激活函数之后的输出的影响。

回顾：激活函数 $\sigma(\cdot)$ 的输入输出都是标量。

$$\boldsymbol{a}^L = \sigma(\boldsymbol{z}^L) = \sigma(\boldsymbol{w}^L\boldsymbol{a}^{L-1} + \boldsymbol{b}^L)$$



反向传播的四个重要公式（一）

- $$\delta_j^L = \frac{\partial C}{\partial a_j^L}\sigma'(z_j^L)$$

这个公式中的各部分都很容易计算。假如

$$C = \frac{1}{2}\sum_j (y_j - a_j^L)^2,$$

那么

$$\frac{\partial C}{\partial a_j^L} = a_j^L - y_j$$

反向传播的四个重要公式（一）

- $$\delta_j^L = \frac{\partial C}{\partial a_j^L}\sigma'(z_j^L)$$

公式表示的是 $\boldsymbol{\delta}^L$ 中的一个元素的情况。若要写成向量的形式，需要用到我们刚定义的哈达玛积

$$\boldsymbol{\delta}^L = \nabla_a C \odot \sigma'(\boldsymbol{z}^L),$$

其中，$\nabla_a C$ 为向量，其各元素为 $\frac{\partial C}{\partial a_j^L}$，$\nabla_a C$ 表示了 C 相对于神经网络输出 $\boldsymbol{a}^L$ 的变化率

反向传播的四个重要公式（一）

- $$\boldsymbol{\delta}^L = \nabla_a C \odot \sigma'(\boldsymbol{z}^L),$$

考虑我们用过的例子，

$$C = \frac{1}{2}\sum_j (y_j - a_j^L)^2,$$

那么

$$\nabla_a C = \boldsymbol{a}^L - \boldsymbol{y},$$

基于这个代价函数，公式一的形式为

$$\boldsymbol{\delta}^L = (\boldsymbol{a}^L - \boldsymbol{y}) \odot \sigma'(\boldsymbol{z}^L).$$

反向传播的四个重要公式（二）：下一层→当前层

- 第二个公式：本层误差量与下一层的误差量的关系

$$\boldsymbol{\delta}^l = ((\boldsymbol{w}^{l+1})^T \boldsymbol{\delta}^{l+1}) \odot \sigma'(\boldsymbol{z}^l)$$

这个公式有直观的解读：假设我们知道了第 $l+1$ 层的误差量 $\boldsymbol{\delta}^{l+1}$，我们让这个误差量“反向通过”神经网络，即用该层权重组成的矩阵的转置 $(\boldsymbol{w}^{l+1})^T$ 去乘以这个误差量，就可以得到与上一层误差量相关的量

这个公式可用微积分链式法则推得。回顾：

$$\delta_j^l = \frac{\partial C}{\partial z_j^l} \qquad \boldsymbol{a}^l = \sigma(\boldsymbol{z}^l) = \sigma(\boldsymbol{w}^l \boldsymbol{a}^{l-1} + \boldsymbol{b}^l)$$

反向传播的四个重要公式（二）

- $$\boldsymbol{\delta}^L = \nabla_a C \odot \sigma'(\boldsymbol{z}^L),$$

$$\boldsymbol{\delta}^l = ((\boldsymbol{w}^{l+1})^T \boldsymbol{\delta}^{l+1}) \odot \sigma'(\boldsymbol{z}^l)$$

有了公式一和公式二，我们就可以计算任意一层的“误差量”

反向传播的四个重要公式（三）：偏置→损失函数

- 第三个公式：损失函数改变率与偏置

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l$$

其中，$\delta_j^l = \frac{\partial C}{\partial z_j^l}$，且 $z_j^l = \sum_k w_{jk}^l a_k^{l-1} + b_j^l$

写成向量的形式

$$\frac{\partial C}{\partial \boldsymbol{b}^l} = \boldsymbol{\delta}^l$$

反向传播的四个重要公式（四）：权重→损失函数

- 第四个公式：损失函数改变率与权重

$$\frac{\partial C}{\partial w_{jk}^{l}} = a_{k}^{l-1}\delta_{j}^{l}$$

其中，$\delta_{j}^{l} = \frac{\partial C}{\partial z_{j}^{l}}$，且 $z_{j}^{l} = \sum_{k} w_{jk}^{l} a_{k}^{l-1} + b_{j}^{l}$

我们也可以把这个公式简写成

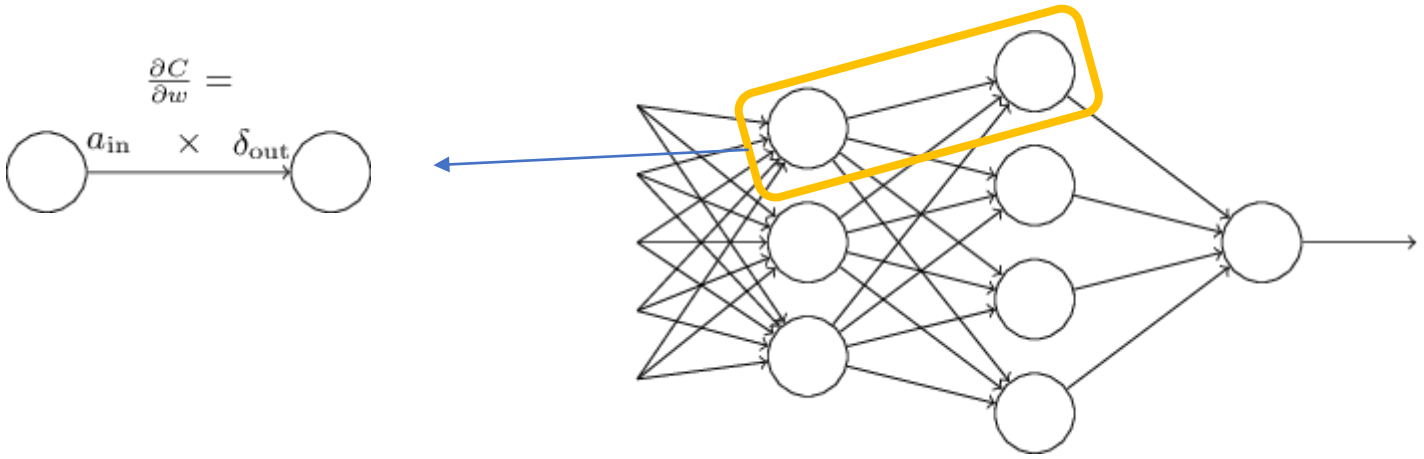
$$\frac{\partial C}{\partial w} = a_{in}\delta_{out},$$

其中 a_{in} 是输入到这个神经元的这个权重下的信息，即上一层的一个输出；δ_{out} 是当前个神经元的误差量

反向传播的四个重要公式（四）

- $$\frac{\partial C}{\partial w} = a_{in}\delta_{out},$$

从整个神经网络中截取一部分，我们可以把上述公式看作



反向传播的四个重要公式（四）

- $$\frac{\partial C}{\partial w} = a_{in}\delta_{out},$$

  这个式子也说明，若 $a_{\in} \approx 0$，那么偏导 $\frac{\partial C}{\partial w}$ 也容易较小。这个时候，权重 w 的学习就会较慢，即在梯度下降中，变化不大。

  换句话说，在输出较小的神经元中，权重的学习常常更慢。

反向传播的四个重要公式

- $$\boldsymbol{\delta}^L = \nabla_a C \odot \sigma'(\boldsymbol{z}^L),$$

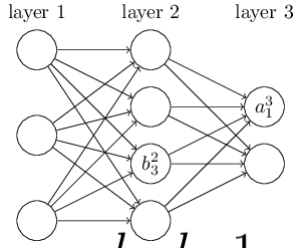
$$\boldsymbol{\delta}^l = ((\boldsymbol{w}^{l+1})^T \boldsymbol{\delta}^{l+1}) \odot \sigma'(\boldsymbol{z}^l)$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l$$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l$$

根据以上的四个重要公式，我们可以计算梯度 $\frac{\partial C}{\partial \boldsymbol{w}}$ 和 $\frac{\partial C}{\partial b}$，具体算法见下页

反向传播算法



- 1. 输入 x：设置网络第一层 $\boldsymbol{a}^1$ 的值

  2. 前馈：对每一层 $l = 2, 3, ..., L$ 计算 $\boldsymbol{z}^l = \boldsymbol{w}^l \boldsymbol{a}^{l-1} + \boldsymbol{b}^l$ 以及 $\boldsymbol{a}^l = \sigma(\boldsymbol{z}^l)$

  3. 输出端“误差量” $\boldsymbol{\delta}^L$：计算最后一层的“误差量” $\boldsymbol{\delta}^L = \nabla_a C \odot \sigma'(\boldsymbol{z}^L)$

  4. “误差量”反向传播：对每一层 $l = L-1, L-2, ..., 2$，计算 $\boldsymbol{\delta}^l = ((\boldsymbol{w}^{l+1})^T \boldsymbol{\delta}^{l+1}) \odot \sigma'(\boldsymbol{z}^l)$

  5. 梯度：计算所需的梯度 $\frac{\partial C}{\partial w^l_{jk}} = a^{l-1}_k \delta^l_j$ 和 $\frac{\partial C}{\partial b^l_j} = \delta^l_j$

反向传播算法 + mini-batch

- 上页描述的算法适合基于一个输入 x 的损失函数 $C = C_x$ 计算相关梯度

  回顾：实际训练时，我们常常使用 mini-batch，即一次对 m 个样本 x，计算 $C = \frac{1}{n}\sum_x C_x$ 的梯度，使用随机梯度下降法来训练神经网络的参数

反向传播算法 + mini-batch + 随机梯度下降

- 1. 输入 m 个训练样本（一个 mini-batch）

  2. 对每个训练样本 x，按前述的反向传播算法计算 $\boldsymbol{\delta}^{x,l} = \left((\boldsymbol{w}^{x,l+1})^T \boldsymbol{\delta}^{x,l+1}\right) \odot \sigma'(\boldsymbol{z}^{x,l})$

  3. 随机梯度下降：对每一层 $l = L, L-1, ..., 2$，按照以下方式更新权重和偏置

$$\boldsymbol{w}^l \to \boldsymbol{w}^l - \frac{\eta}{m} \sum_x \boldsymbol{\delta}^{x,l} (\boldsymbol{a}^{x,l-1})^T$$

$$\boldsymbol{b}^l \to \boldsymbol{b}^l - \frac{\eta}{m} \sum_x \boldsymbol{\delta}^{x,l}$$

  其中，$\eta > 0$

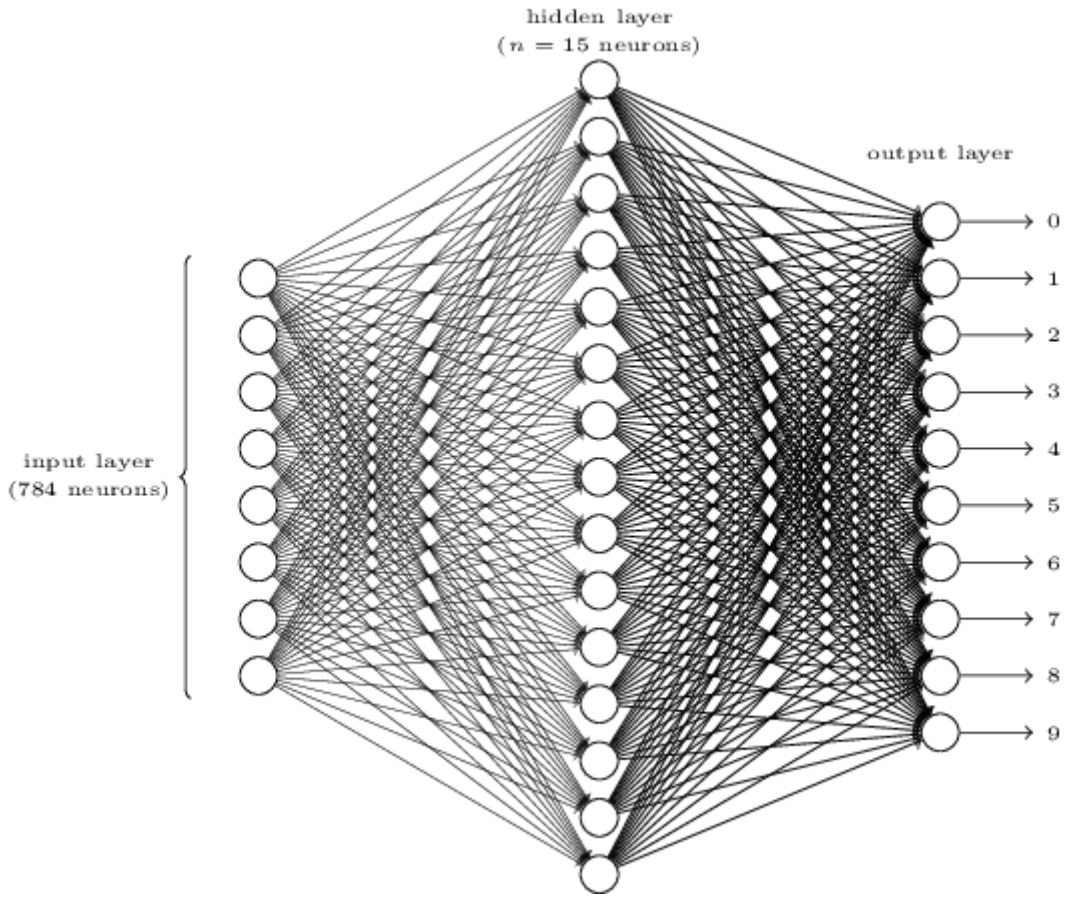
反向传播算法 + mini-batch + 随机梯度下降

- 上页的算法是针对一个 mini-batch 的

  实际在训练神经网络时，还需要在上页的算法之外再加一层循环：即使用若干 mini-batches 来训练

PyTorch

在 PyTorch 中实现神经网络



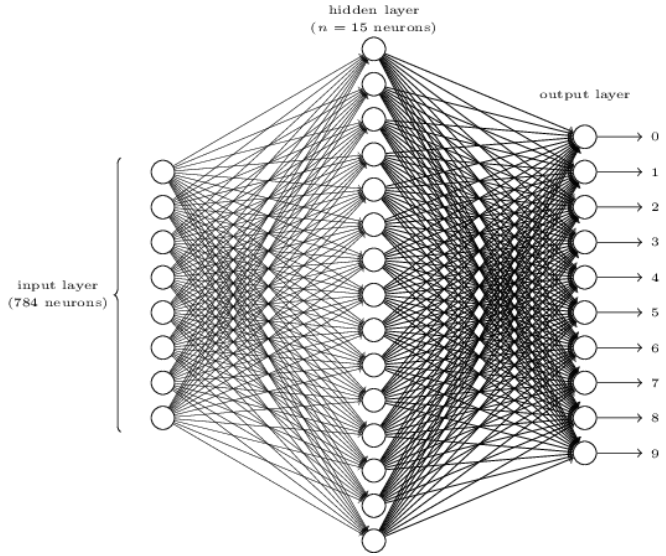
在 PyTorch 中实现神经网络

```
import torch
import torch.nn as nn

class NeuralNet(nn.Module):
    def __init__(self):
        super(NeuralNet, self).__init__()
        self.fc1 = nn.Linear(784, 15)
        self.sigmoid = nn.Sigmoid()
        self.fc2 = nn.Linear(15, 10)

    def forward(self, x):
        out = self.fc1(x)
        out = self.sigmoid(out)
        out = self.fc2(out)
        return out
```

设计模型结构



在 PyTorch 中实现神经网络

- 选择计算硬件（GPU vs CPU）
- 建立模型实例
- 选择损失函数和优化器（包含优化器学习率参数）

```
device = torch.device('cuda' if torch.cuda.is_available() else
'cpu')

model = NeuralNet().to(device)

# Loss and optimizer

criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.001)
```

在 PyTorch 中实现神经网络

- **神经网络训练（单步，即单个batch）**

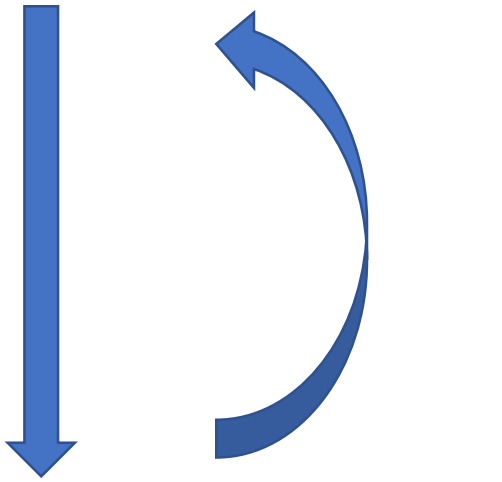
```
# batch 数据样本特征：images；batch 数据样本标签：labels
# Forward pass

outputs = model(images)
loss = criterion(outputs, labels)

# Backward and optimize

optimizer.zero_grad() # 梯度清零
loss.backward()
optimizer.step()
```

下一个 mini-batch，循环进行



在 PyTorch 中实现神经网络

- **定义神经网络结构，建立 model**
- **选择损失函数 criterion**
- **选择优化器 optimizer**

- **此外，还需要定义每个 mini-batch，确定停止训练的条件**（前述代码尚未包含这部分）
  - 详见下页

在 PyTorch 中实现神经网络

In [1]:
```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
```

In [2]:
```
# Device configuration
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

In [3]:
```
# Hyper-parameters
input_size = 784
hidden_size = 50
num_classes = 10
num_epochs = 25
batch_size = 100
learning_rate = 0.001
```

In [4]:
```
# MNIST dataset
train_dataset = torchvision.datasets.MNIST(root='data',
                                           train=True,
                                           download=True,
                                           transform=transforms.ToTensor())

test_dataset = torchvision.datasets.MNIST(root='data',
                                          train=False,
                                          transform=transforms.ToTensor(),
                                          download=True)

# Data loader
train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
                                           batch_size=batch_size,
                                           shuffle=True)

test_loader = torch.utils.data.DataLoader(dataset=test_dataset,
                                          batch_size=batch_size,
                                          shuffle=False)
```

在 PyTorch 中实现神经网络

In [6]:
```
# Fully connected neural network with one hidden layer
class NeuralNet(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(NeuralNet, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.sigmoid = nn.Sigmoid()
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        out = self.fc1(x)
        out = self.sigmoid(out)
        out = self.fc2(out)
        return out

model = NeuralNet(input_size, hidden_size, num_classes).to(device)
```

In [7]:
```
# Loss and optimizer
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

在 PyTorch 中实现神经网络

In [8]:

```
# Train the model
total_step = len(train_loader)
for epoch in range(num_epochs):
    for i, (images, labels) in enumerate(train_loader):
        # Move tensors to the configured device
        images = images.reshape(-1, 28*28).to(device)
        labels = torch.nn.functional.one_hot(labels,num_classes=10).float().to(device)

        # Forward pass
        outputs = model(images)
        loss = criterion(outputs, labels)

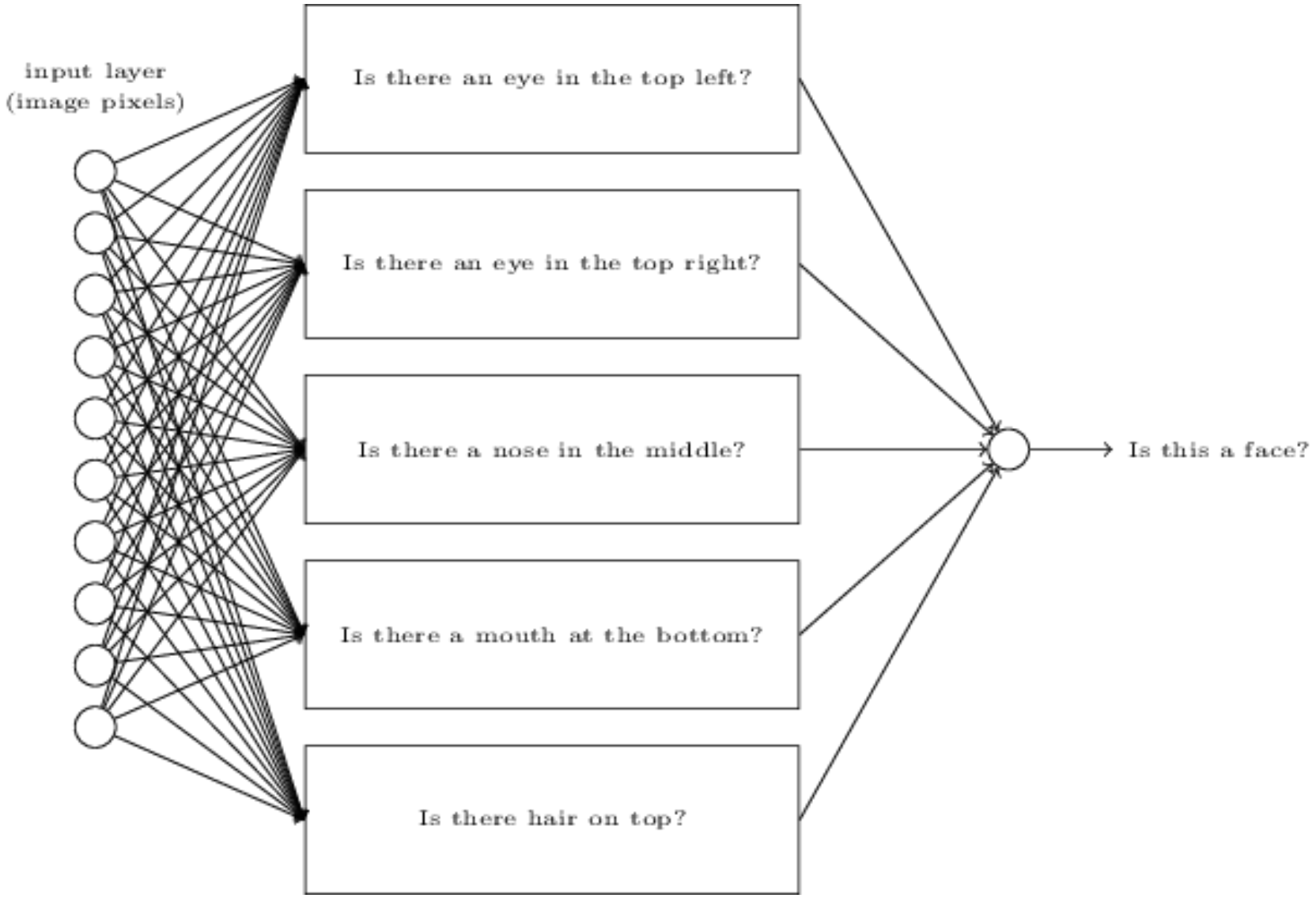
        # Backward and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if (i+1) % 1000 == 0:
            print ('Epoch [{}/{}], Step [{}/{}], Loss: {:.4f}'
                   .format(epoch+1, num_epochs, i+1, total_step, loss.item()))
    # Test the model
    # In test phase, we don't need to compute gradients (for memory efficiency)
    with torch.no_grad():
        correct = 0
        total = 0
        for images, labels in test_loader:
            images = images.reshape(-1, 28*28).to(device)
            labels = labels.to(device)
            outputs = model(images)
            # _, predicted = torch.max(outputs.data, 1)
            predicted = torch.argmax(outputs.data,dim=1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

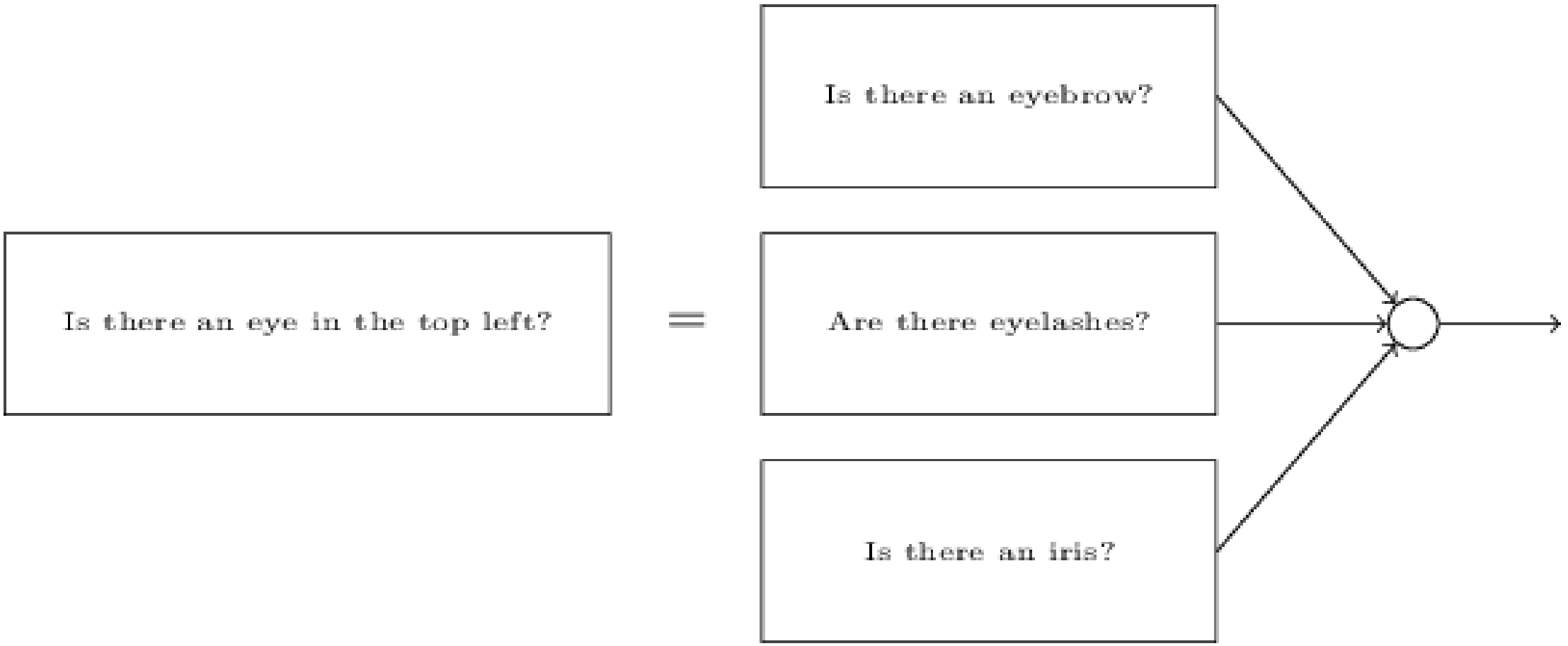
        print('Accuracy of the network on the 10000 test images: {} %'.format(100 * correct / total))
```

```
Accuracy of the network on the 10000 test images: 90.15 %
Accuracy of the network on the 10000 test images: 92.44 %
Accuracy of the network on the 10000 test images: 93.48 %
Accuracy of the network on the 10000 test images: 94.08 %
```

神经元的作用（示意图）


input layer
(image pixels)
Is there an eye in the top left?
Is there an eye in the top right?
Is there a nose in the middle?
Is there a mouth at the bottom?
Is there hair on top?
Is this a face?

神经元的作用（示意图）


Is there an eye in the top left?
=
Is there an eyebrow?
Are there eyelashes?
Is there an iris?